

Python *for AI.*

Read the code. Follow the values.

Build a small program you can explain and test.

🕒 25-30 min core reading · Beginner friendly · 12 short sections

Python gives you a way to turn an AI idea into instructions you can run, inspect, and test. Start with ordinary code that makes every step visible.

WHAT YOU WILL BE ABLE TO DO

Run a Python script, work with values and collections, write a loop and a function, read a small dataset, and save a result. Use those pieces to evaluate the runtime predictions from Chapter 3.

Reading path: Follow the small examples, trace the loop, then run the starter project. Extra syntax and environment setup are expandable. Allow additional time to type and change the code; reading alone is not the practice.

1. Write a file; run it with Python

A **program** is a set of instructions. A Python **interpreter** is the program that executes Python code. A **script** is a text file containing that code, normally saved with a `.py` extension.

On Windows, open a terminal such as PowerShell and check:

```
py --version
```

You need Python 3. If the command is unavailable, follow the [official Windows setup guide](#). If your working command is `python` or `python3`, substitute it for `py` below. A terminal command goes in the terminal; Python code goes in the script.

Create a folder for your practice. Save this as `hello_ai.py` using a code editor or a plain-text editor:

```
job_size = 2.0
weight = 2.0
prediction = weight * job_size + 1.0
print(f"Predicted: {prediction:.2f} s")
```

Open a terminal in that folder and run `py hello_ai.py`. You should see **Predicted: 5.00 s**. Save edits before running again, and check that the file is not accidentally named `hello_ai.py.txt`.

The script calculates a prediction with a chosen weight. It does not learn that weight. Training would need the update process from Chapter 3.

➤ From saved instructions to a visible result



Figure 1. Editing the file changes the saved instructions. Running the interpreter executes them; print displays selected values.

2. Values, names, and types

In `weight = 2.0`, the name `weight` refers to the value `2.0`. Assignment uses `=`; comparing two values for equality uses `==`.

Type	Example	Use in our project
Integer: <code>int</code>	<code>3</code>	Number of jobs
Floating-point number: <code>float</code>	<code>2.5</code>	Size, runtime, error
String: <code>str</code>	<code>"authentication"</code>	Text or a category name
Boolean: <code>bool</code>	<code>True</code> , <code>False</code>	A condition's result
Missing-value marker: <code>None</code>	<code>None</code>	An explicitly absent value

Python values have types, even though you usually do not declare a variable's type in advance. The string `"2.5"` is different from the number `2.5`. Convert suitable numeric text with `float("2.5")`; text such as `"unknown"` cannot be converted that way.

Use `+`, `-`, `*`, and `/` for arithmetic. Exponentiation uses `**`, so `error ** 2` squares an error. Parentheses control grouping.

The earlier `f"..."` is a formatted string: braces insert a value, and `:.2f` displays two decimal places. Formatting changes the displayed text, not the number used in calculations. A comment begins with `#` and explains the code without being executed.

▼ Go deeper: strings, precision, and missing values

Methods are operations accessed through an object. For example, `" Login Failed ".strip().lower()` produces `"login failed"`: remove surrounding whitespace, then make the text lowercase. Strings are immutable; these operations return strings rather than editing the original one.

Most decimal fractions are approximations in binary floating-point arithmetic. For calculated floats, use an appropriate tolerance, such as `math.isclose(a, b)`, instead of assuming exact equality. Keep full precision for calculations and round for display.

`None`, zero, an empty string, and `False` have different meanings even though each is false in a condition. Test `value is None` when you specifically mean "missing."

3. Organize examples with lists and dictionaries

A **list** holds an ordered sequence. A **dictionary** associates keys with values. Together, they give us a readable version of Chapter 3's dataset:

```
jobs = [
    {"size": 1.0, "actual": 3.0},
    {"size": 2.0, "actual": 5.0},
    {"size": 3.0, "actual": 7.0},
]
print(jobs[0]["size"]) # 1.0
print(len(jobs))      # 3
```

List positions start at zero: `jobs[0]` is the first row. That row is a dictionary, so `["size"]` retrieves the value under its `"size"` key. The built-in function `len` returns the number of items.

➤ A list contains rows; a dictionary names fields



Figure 2. Read the expression from left to right: select row zero, then retrieve its size field. Index zero is the first item.

Use `jobs.append(new_job)` to add one row. `jobs[:2]` selects the first two rows; the stop position is excluded. A missing dictionary key raises `KeyError`. Use `row.get("note")` for an optional field that may return `None`, but do not quietly treat a missing required runtime as zero.

▼ Go deeper: copying, tuples, sets, and comprehensions

Lists and dictionaries are mutable: they can change. Assignment does not automatically copy them.

```
original = [1, 2]
alias = original
alias.append(3)
print(original) # [1, 2, 3]
```

`original.copy()` creates a new outer list. For a list of dictionaries, the nested dictionaries are still shared; changing a nested value can affect both lists. This matters when separating original data from prepared data.

A **tuple**, such as `(1.0, 2.0)`, is an ordered collection whose slots cannot be reassigned. A **set** keeps unique elements and supports membership checks; it does not promise list-style order or indexing.

A **list comprehension** constructs a list from an expression and iteration. After defining the prediction function in Section 5, this collects three predictions:

```
predictions = [predict(row["size"], 2.0) for row in jobs]
print(predictions) # [3.0, 5.0, 7.0]
```

Prefer the longer loop until you can explain what the shorter expression does.

4. Choose a branch; repeat a calculation

An `if` statement chooses a branch from a condition. Indentation defines which instructions belong together; use four spaces consistently.

```
prediction = 5.0
if prediction > 6:
    print("Review the estimate")
else:
    print("Within the example limit")
```

This prints **Within the example limit**. Conditions can use comparisons such as `>`, `<=`, and `==`, combined with `and`, `or`, and `not`.

A `for` loop repeats a block for each item. Keep the `jobs` list from Section 3 and run:

```
weight = 1.0
total = 0.0
for row in jobs:
    prediction = weight * row["size"] + 1.0
    error = prediction - row["actual"]
    total += error ** 2

mse = total / len(jobs)
print(f"MSE: {mse:.4f} s^2")
```

The output is **MSE: 4.6667 s²**. Here `+=` adds to the current total. Put the initialization before the loop; otherwise it resets for every row. Put the final average after the loop, when all rows have contributed.

A `while` loop instead repeats while a condition remains true. Its body must eventually change something that makes the condition false, or deliberately stop. We use `for` here because we already have a finite collection.

5. Give a reusable calculation a function

A **function** is a named operation that can receive inputs and return a result:

```
def predict(size, weight):  
    return weight * size + 1.0  
  
estimate = predict(2.5, 2.0)  
print(estimate) # 6.0
```

`size` and `weight` are **parameters** in the definition. `2.5` and `2.0` are **arguments** supplied by the call. `return` sends the result back to the caller and ends that function call.

`print` displays something; it does not replace `return`. A function that reaches its end without returning a value returns `None`. Keep calculation separate from display so you can reuse and test the calculation.

A function's local names belong to that call. Passing values explicitly is easier to follow than relying on a changing global variable. Replace the prediction expression in the previous loop with `predict(row["size"], weight)`.

▼ Go deeper: type hints, classes, and model objects

A type hint can document an expected input and result:

```
def predict(size: float, weight: float) -> float:  
    return weight * size + 1.0
```

Type hints help readers and tools; Python does not automatically validate every value against them at runtime. Input checks are a separate responsibility.

A **class** defines a kind of object with stored state and operations called methods. This small class stores a weight:

```
class RuntimeModel:  
    def __init__(self, weight):  
        self.weight = weight  
  
    def predict(self, size):  
        return self.weight * size + 1.0  
  
model = RuntimeModel(2.0)  
print(model.predict(2.5)) # 6.0
```

`self` refers to the instance on which the method is called. `__init__` initializes its state. Libraries often expose model objects with methods, but naming an object "model" does not train it. This class only stores and uses the supplied weight.

6. Trace the loop before changing it

The arithmetic is familiar; the new skill is tracking the program's state. At $w = 1$, the squared errors are 1, 4, and 9. The running total becomes 1, then 5, then 14.

FOLLOW THE VALUES

Recorded Python execution

One row at a time.

These snapshots come from the supplied Python program. The controls replay results; your browser does not execute Python.

At $w = 1$, the recorded loop produces:

Row	Prediction	Squared error	Running total
1	2.0	1.0	1.0
2	3.0	4.0	5.0
3	4.0	9.0	14.0

Final MSE = $14 / 3 = 4.6667 s^2$. At $w = 2$, predictions are 3, 5, and 7; all errors and the final MSE are zero.

Before advancing each row, predict the next total yourself. After the final row, divide by three to obtain MSE. Compare the recorded run at $w = 2$: changing a supplied parameter changes predictions, but this evaluator still does not perform training.

The website replays results produced by the downloadable Python example. To edit and execute Python, use the starter project in Section 9.

7. Imports, modules, and environments

A **module** groups Python code under a name. `import` makes its functionality available; it does not install a missing library.

```
import math

print(math.isclose(0.1 + 0.2, 0.3)) # True
```

`math`, `json`, and `pathlib` are in Python's **standard library**, supplied with Python. Our starter needs no extra packages. A **package** groups importable modules; third-party libraries are usually installed as distributions using a tool such as `pip`.

An **environment** determines which interpreter and installed packages a project uses. A virtual environment helps separate one project's dependencies from another's.

▼ Practical setup: use a virtual environment on Windows

Run these terminal commands from your project folder:

```
py -m venv .venv
.\.venv\Scripts\python.exe evaluate_jobs.py
.\.venv\Scripts\python.exe -m pip --version
```

The first creates the environment; the second runs the starter with its interpreter. Using that interpreter's full relative path avoids needing to activate the environment. `-m` runs a module, so the last command checks pip for that same interpreter.

For future chapters, install dependencies with the chosen interpreter's `-m pip install` command. Record the versions you use so a project can be recreated. See [Python's virtual-environment guide](#).

Different tools serve different needs: **NumPy** provides numerical arrays, **pandas** works with tables, **scikit-learn** supplies ML estimators, and **PyTorch** supports tensor computation and neural networks. We will introduce them when needed. A normal Python list is not a NumPy array; list multiplication repeats its contents rather than multiplying every element.

8. Read a file; handle a failure clearly

JSON is a text format for structured data. It resembles parts of Python syntax but is not Python code: JSON uses double-quoted keys and `true`, `false`, and `null` instead of Python's corresponding values.

With the starter files in the same folder, this script fragment loads the dataset:

```
import json
from pathlib import Path

folder = Path(__file__).resolve().parent
text = (folder / "jobs.json").read_text(encoding="utf-8")
jobs = json.loads(text)
```

`Path` represents a filesystem path. Here `file` identifies the script, and `.parent` identifies its containing folder. The path's `/` operation joins a filename to the folder. Reading produces text; `json.loads` turns that text into Python values such as a list of dictionaries. Parsing JSON does not prove that required fields and types are correct.

Save a Python result by converting it back to JSON text:

```
result = {"weight": 2.0, "mse": 0.0}
output = folder / "example_result.json"
output.write_text(json.dumps(result, indent=2), encoding="utf-8")
```

This creates or replaces that output file. The values shown here are an illustration; the starter calculates its report from the loaded rows.

When execution fails, Python usually shows a **traceback**: the call path, the location, and the exception. Read the final error message, then locate the relevant line in your file.

Error	What to check
<code>SyntaxError</code> or <code>IndentationError</code>	Missing colon, unmatched bracket, or incorrect indentation
<code>NameError</code>	A name is misspelled or used before assignment
<code>TypeError</code>	An operation received an unsuitable type
<code>KeyError</code>	A required dictionary key is absent
<code>FileNotFoundError</code>	The path does not point to an existing file

▼ Go deeper: catch expected errors, then explain them

Use `try` and `except` when you have a useful response to a specific failure:

```
raw_size = "unknown"
try:
    size = float(raw_size)
except ValueError:
    print("Job size must be numeric")
```

The code reports a conversion problem. It does not continue using a fabricated zero. Avoid silently ignoring every exception, because that can hide programming mistakes.

Use `raise ValueError("jobs cannot be empty")` to reject invalid input explicitly. The starter checks for an empty dataset and unsuitable numeric fields before calculating a score.

For lower-level file handling, `with open(...)` closes the file when the block exits, including when an exception occurs. The `Path` convenience methods above manage their file access for you. See [Python's exceptions guide](#).

9. Build a small prediction evaluator

Download and extract the starter. It contains `evaluate_jobs.py`, `jobs.json`, and instructions.

The starter ZIP is available through the Download Python starter link on Chapter 4's website.

From the extracted folder, run `py evaluate_jobs.py`. The expected terminal output is:

```
w=1.0 | jobs=3 | MSE=4.6667 s^2
w=2.0 | jobs=3 | MSE=0.0000 s^2
Saved runtime_report.json
```

➤ Keep the data, calculation, and report distinct



Figure 3. The starter reads its input beside the script, checks the rows, evaluates fixed weights, then creates or replaces its generated report.

Read the three functions in the downloaded file:

predict: computes one runtime from size and weight.

evaluate: checks the rows, loops over them, records errors, and returns a dictionary containing MSE and the row results.

main: reads the input file, evaluates two chosen weights, prints the summary, and writes `runtime_report.json` beside the script.

The final `if name == "main":` block runs the entry point when the file is executed directly. Importing the module makes its functions available without automatically running that entry point.

Inspect the generated JSON. At $w = 1$, row C should have prediction 4, error -3 , squared error 9, and running total 14. Running the script again replaces its generated report.

Add a tiny check in a separate `check_example.py` beside the starter:

```
from evaluate_jobs import predict

assert predict(2.5, 2.0) == 6.0
print("Check passed")
```

`assert` raises an error when the condition is false. This checks one known case; it is not proof of general correctness. Assertions are useful in tests, but use explicit validation for required input checks because assertions can be disabled.

▼ Working habit: scripts, notebooks, and reproducibility

A script normally runs top to bottom in a fresh process. A notebook groups code into cells and keeps state in its running kernel. Running cells out of order can leave hidden dependencies on old values.

Before trusting a notebook, restart its kernel and run the cells in order. Record the input data and environment. For this chapter, a script makes execution order explicit.

If you move the file-reading snippet into a notebook, `file` is normally unavailable. Use an explicit data path or a deliberately chosen working directory instead of assuming the script snippet applies unchanged.

10. Practice: change, predict, verify

Exercise 1: change a data point

In `jobs.json`, change the third actual runtime from 7.0 to 8.0. Before running the script, calculate the new MSE at $w = 2$.

▼ Reveal the calculation

The predictions remain 3, 5, and 7. The errors become 0, 0, and -1 , so MSE is $1 / 3 \approx 0.3333$ squared seconds. The parameter did not change; you changed the measured target. Restore the original data before comparing with the chapter's recorded trace.

Exercise 2: find the misplaced line

Someone puts `total = 0.0` inside the `for` loop immediately before the error calculation. At $w = 1$, what final MSE will the code report, and why is it wrong?

▼ Reveal the explanation

Each iteration discards the earlier total. After the third row, only its squared error of 9 remains. Dividing by three gives 3.0, rather than $14 / 3 \approx 4.6667$. Initialize the accumulator before entering the loop.

Exercise 3: return a reusable result

Write a function `squared_error(predicted, actual)` that returns the squared difference. Check it with `predicted = 4` and `actual = 7`.

▼ Reveal the solution

```
def squared_error(predicted, actual):  
    return (predicted - actual) ** 2  
  
assert squared_error(4, 7) == 9
```

The function returns 9. Printing 9 inside the function without returning it would leave the caller with `None`.

11. Check your understanding

Answer all five questions, then review the explanation for any distinction you missed.

01 What does `jobs[0]["size"]` mean?

- A. The size field in the first row.
- B. The number of rows in jobs.
- C. The size field in the last row.

Answer: A. List indexing starts at zero. Selecting `jobs[0]` gives the first dictionary, then the `size` key retrieves its value.

02 A function prints 6.0 but has no return statement. What does assigning its result give you?

- A. The string "6.0".
- B. The number 6.0.
- C. None.

Answer: C. Displaying a value is different from returning it. A function that reaches its end without returning a value returns `None`.

03 Where should `total = 0.0` go when accumulating squared errors?

- A. Inside the loop before every addition.
- B. Before the loop begins.
- C. After calculating the final average.

Answer: B. Initialize once before the loop. Initializing inside the loop discards contributions from earlier rows.

04 What does `json.loads(text)` establish?

- A. It parsed valid JSON into Python values; fields and types still need checking.
- B. The data contains every required field.
- C. The contents are suitable for training a reliable model.

Answer: A. Successful parsing checks the JSON format, not the application schema, data quality, or suitability for model training.

05 The starter compares MSE at weights 1 and 2. What is it doing?

- A. Learning the weights using gradient descent.
- B. Evaluating two supplied weights on the example data.
- C. Proving future predictions will be correct.

Answer: B. The program receives fixed candidate weights and calculates errors. It contains no optimizer update, and its tiny dataset does not establish generalization.

12. Keep a small working toolkit

You can now follow the path from a data file to Python values, through functions and loops, to a checked output. Explain that path before adding a model library.

▼ Quick-reference glossary

Term	Recall cue
Interpreter / script	Program executing Python / file containing Python code
Variable / type	A name referring to a value / the kind of that value
List / dictionary	Ordered collection / keys associated with values
Condition / loop	A decision / repeated execution
Parameter / argument	Input name in a function definition / value supplied in a call
Return / print	Send a value to the caller / display text
Module / import	Organized code / make it available by name
Virtual environment	An interpreter environment with project-specific packages
JSON	Text representation of structured data
Exception / traceback	A reported failure / execution context showing where it arose
Assertion	A condition checked during a test

The dataset is synthetic and the examples are original. Optional references:

Python: values, strings, and lists

Python: control flow and functions

Python: data structures

Python: files and JSON

Next: Chapter 5: Software, APIs, and data fundamentals.